

기초 영상처리

영상 분할 및 특징처리

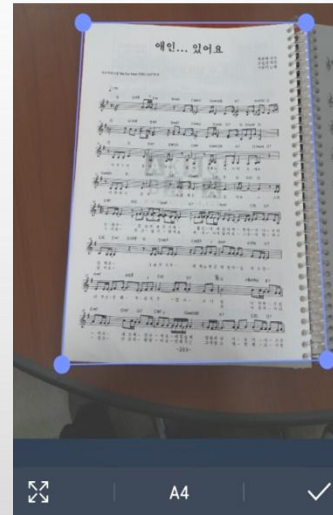
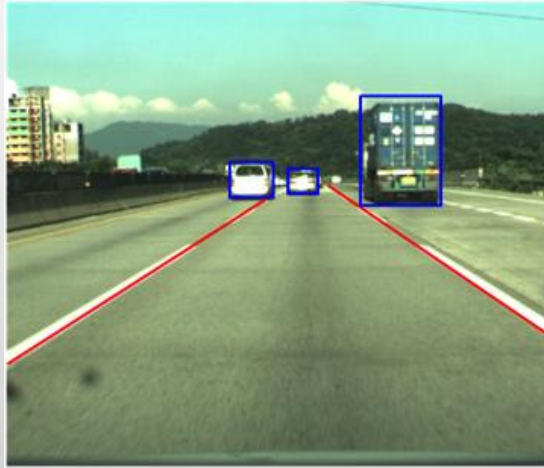
10.1 허프 변환

◆ 직선 검출

- ❖ 영상 내에서 공간 구조를 분석하는데 유용한 도구
- ❖ 영상 처리와 컴퓨터 비전분야에서 많은 연구 진행

◆ 다양한 응용에 사용

- ❖ 차선 및 장애물 자동인식 시스템 - 차선 검출
- ❖ 스캐너의 기능을 대신해 주는 앱 - 네 개 모서리 검출



10.1.1 허프 변환의 좌표계

◆허프변환

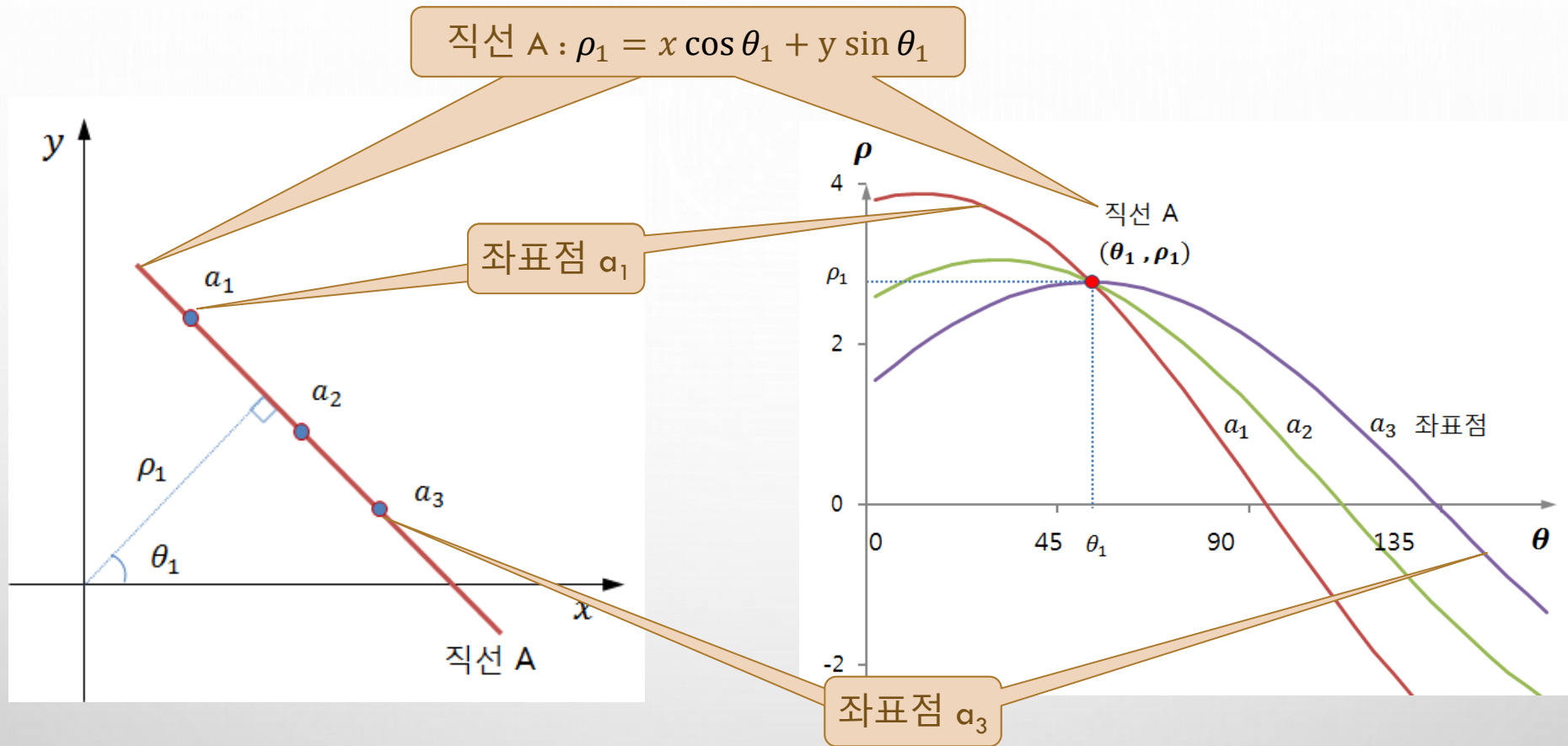
❖ 직교 좌표계로 표현되는 영상의 에지 점들을 극 좌표계로 옮겨, 검출하고자 하는 물체의 파라미터(ρ, θ)를 추출하는 방법

$$y = ax + b \leftrightarrow \rho = x \cdot \cos \theta + y \cdot \sin \theta$$

❖ 직교 좌표계에서 검출의 문제점 (클래식 허프변환)

- 수직선일 경우에 기울기가 무한대
- 검출되는 직선의 간격이 동일하지 않아서 검출 속도와 정밀도에서 문제

10.1.1 허프 변환의 좌표계



- ❖ 직교좌표의 직선은 허프변환 좌표에서 한점 (ρ_1, θ_1) 으로 표현
- ❖ 직교좌표의 한 점은 허프변환 좌표에서 곡선으로 표현

10.1.2 허프 변환의 전체 과정

1. 극 좌표계에서 누적 행렬 구성
2. 영상 화소의 직선 검사
3. 직선 좌표에 대한 극좌표 누적 행렬 구성
4. 누적 행렬의 지역 최댓값 선정
5. 직선 선별 - 임계값 이상인 누적값 선택 및 정렬

◆ 허프 변환 좌표계를 위한 행렬의 계산

$$\begin{aligned} -\rho_{\max} \leq \rho \leq \rho_{\max}, \quad \rho_{\max} = rows + cols, \quad h = \frac{\rho_{\max}}{\Delta \rho} * 2 \\ 0 \leq \theta < \theta_{\max}, \quad \theta_{\max} = \pi, \quad w = \frac{\pi}{\Delta \theta} \end{aligned}$$

교재 오타

각도간격,
거리간격

10.10 허프 누적 행렬 구성

거리간격, 각도간격

누적 행렬

삼각 함수 행렬

직선 극좌표를 누적 행렬의 인덱스로 계산

```
01 def accumulate(image, rho, theta):
02     h, w = image.shape[:2]
03     rows, cols = (h+w) * 2 // rho, int(np.pi / theta)        # 누적행렬 너비, 높이
04     accumulate = np.zeros((rows, cols), np.int32)              # 직선 누적행렬
05
06     sin_cos = [(np.sin(t*theta), np.cos(t*theta)) for t in range(cols)] # 삼각 함수값 저장
07     pts = np.where(image > 0)                                    # 넘파이 함수 활용 - 직선좌표 찾기
08
09     polars = np.dot(sin_cos, pts).T                             # 행렬 곱으로 극좌표 계산
10     polars = (polars / rho + rows / 2).astype('int')           # 해상도 변경 및 위치 조정

    for row in polars:
13         for t, r in enumerate(row):                            # 각도, 수직 거리 가져옴
14             accumulate[r, t] += 1                               # 극좌표에 누적
15     return accumulate
```

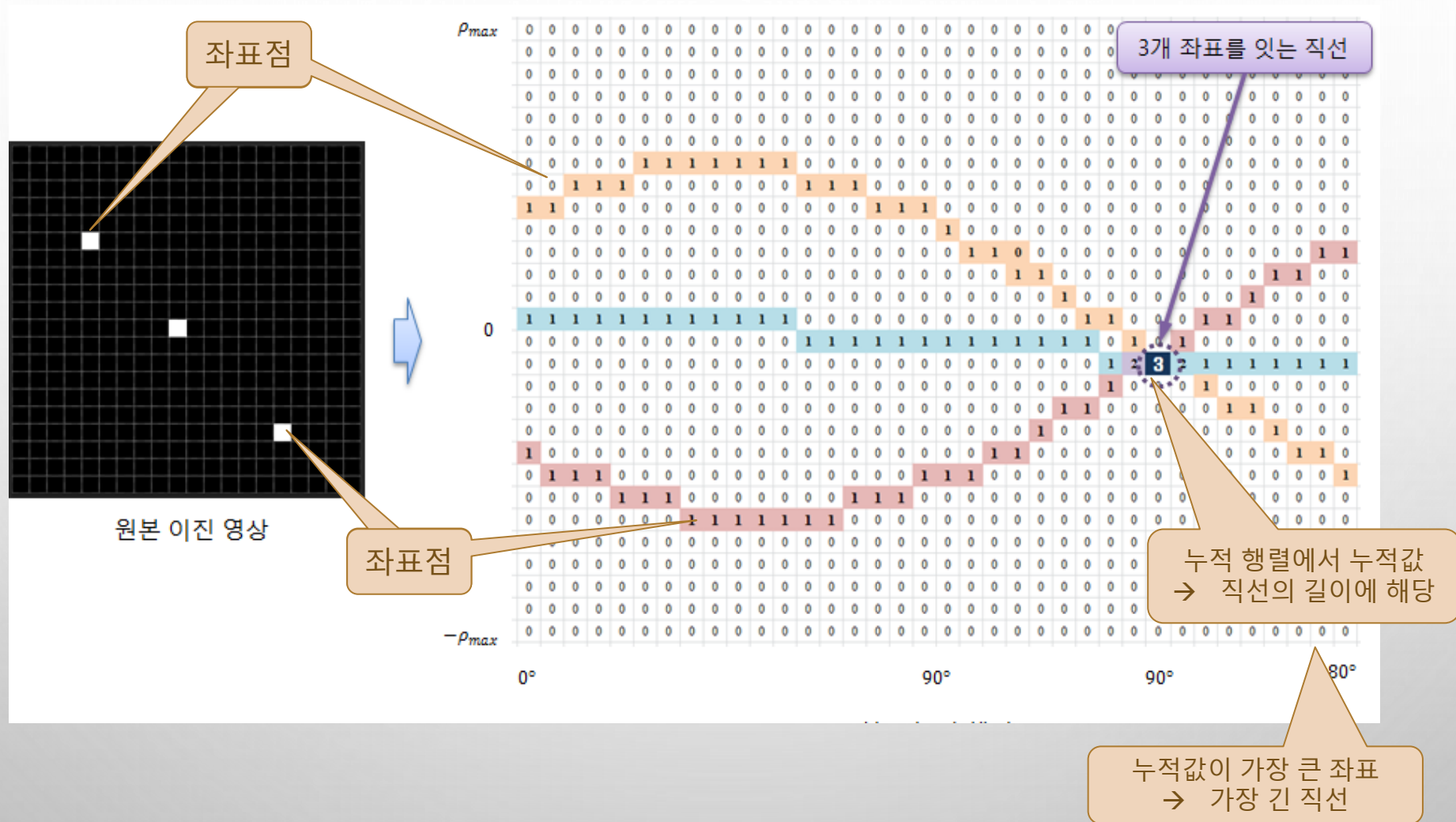
직선 좌표들의 극좌표

0~180도 각도에 대한
삼각함수 값 행렬

$$\begin{bmatrix} \sin \theta_0 & \cos \theta_0 \\ \sin \theta_1 & \cos \theta_1 \\ \sin \theta_2 & \cos \theta_2 \\ \dots & \dots \\ \sin \theta_m & \cos \theta_m \end{bmatrix} \cdot \begin{bmatrix} y_0 & y_1 & y_2 & \dots & y_n \\ x_0 & x_1 & x_2 & \dots & x_n \end{bmatrix} = \begin{bmatrix} \rho_0, \theta_0 & \rho_1, \theta_0 & \rho_2, \theta_0 & \dots & \rho_n, \theta_0 \\ \rho_0, \theta_1 & \rho_1, \theta_1 & \rho_2, \theta_1 & \dots & \rho_n, \theta_1 \\ \rho_0, \theta_2 & \rho_1, \theta_2 & \rho_2, \theta_2 & \dots & \rho_n, \theta_2 \\ \dots & \dots & \dots & \dots & \dots \\ \rho_0, \theta_m & \rho_1, \theta_m & \rho_2, \theta_m & \dots & \rho_n, \theta_m \end{bmatrix}$$

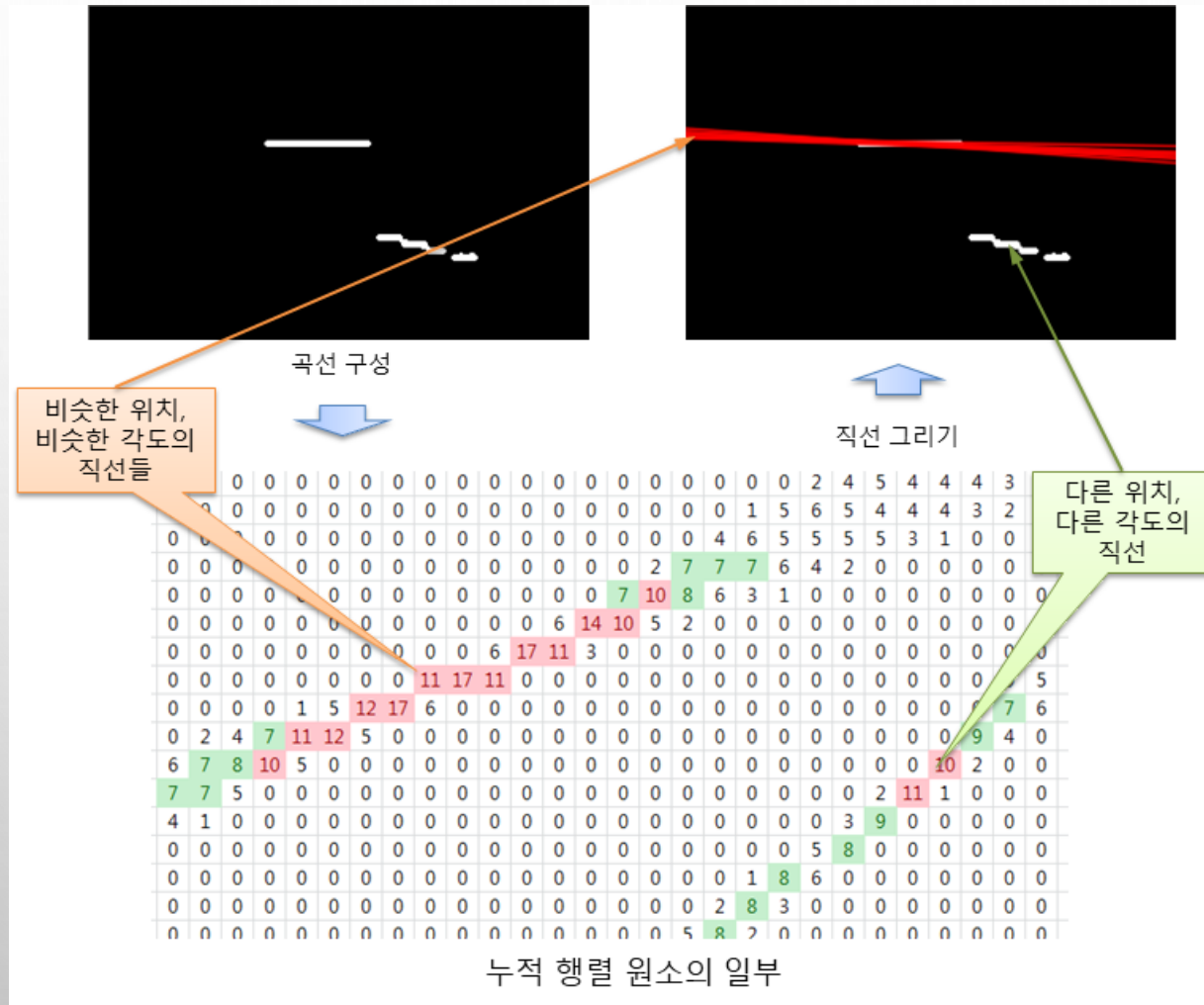
직선 좌표들

10.1.3 허프 누적 행렬 구성



10.1.3 허프 누적 행렬 구성

❖ 한 지점에서 여러 직선 검출의 문제



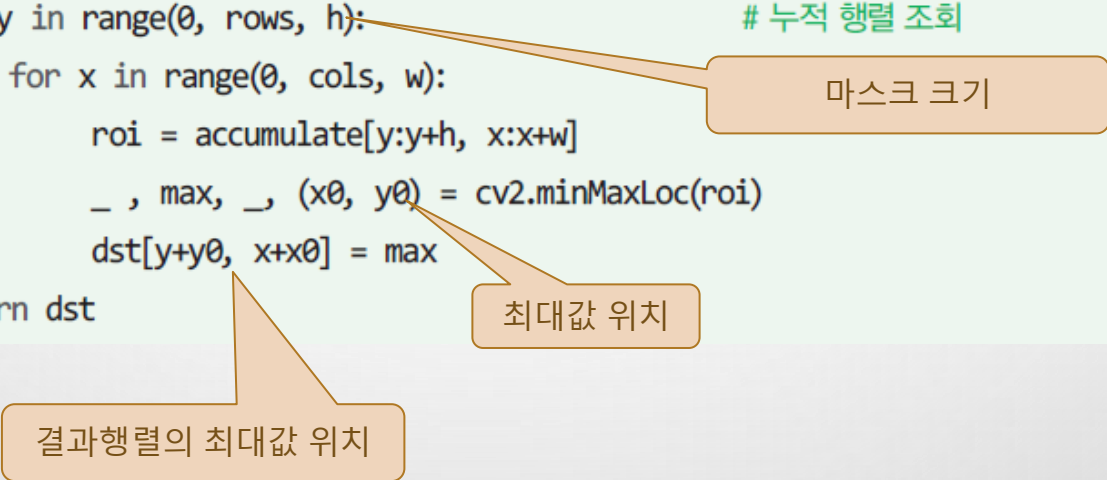
10.1.4 허프 누적 행렬의 지역 최대값 선정

◆ 마스크를 이용해 지역 최대값 선정



10.1.4 허프 누적 행렬의 지역 최대값 선정

```
01 def masking(accumulate, h, w, thresh):
02     rows, cols = accumulate.shape[:2]
03     rcenter, tcenter = h//2, w//2           # 마스크 크기 절반
04     dst = np.zeros(accumulate.shape, np.uint32)
05
06     for y in range(0, rows, h):             # 누적 행렬 조회
07         for x in range(0, cols, w):
08             roi = accumulate[y:y+h, x:x+w]
09             _, max, _, (x0, y0) = cv2.minMaxLoc(roi)
10             dst[y+y0, x+x0] = max
11     return dst
```



마스크 크기

최대값 위치

결과행렬의 최대값 위치

10.1.5 직선(극 좌표) 선택 및 정렬

❖ 직선 선별 함수

극 좌표 행렬의 인덱스

```
01 def select_lines(acc_dst, rho, theta, thresh):  
    rows = acc_dst.shape[0]  
03     r, t = np.where(acc_dst>thresh)  
04  
05     rhos = ((r - (rows / 2)) * rho)  
06     radians = t * theta  
07     values = acc_dst[r, t]  
  
09     idx = np.argsort(values)[::-1]  
10     lines = np.transpose([rhos, radians])  
11     lines = lines[idx, :]  
12  
13     return np.expand_dims(lines, axis=1)
```

누적값 기준 정렬

직선 벡터(lines)에 단일 직선 저장

임계값 이상 인덱스 가져옴

인덱스로 수직 거리 계산

인덱스로 각도 계산

인덱스로 누적값 가져옴

내림차순 정렬 인덱스

리스트 전치하여 행렬 생성

누적값 기준으로 극좌표 정렬

1번(열) 차원 증가

OpenCV 함수와 동일하게 반환 자료형 맞춤

수직거리 행렬

$$\begin{bmatrix} \rho_0 & \rho_1 & \rho_2 & \cdots & \rho_n \\ \theta_0 & \theta_1 & \theta_2 & \cdots & \theta_n \end{bmatrix}$$

각도 행렬

직선 극 좌표

전치

$$\begin{bmatrix} \rho_0 & \theta_0 \\ \rho_1 & \theta_1 \\ \rho_2 & \theta_2 \\ \vdots & \vdots \\ \rho_n & \theta_n \end{bmatrix}$$

직선 극 좌표

10.1.7 최종 완성 프로그램

예제 10.1.1

허프 변환을 이용한 직선 검출 - 01.hough_lines.py

```
01 import numpy as np, cv2, math
02 from Common.hough import accumulate, masking, select_lines    # 허프 변환 함수 импорт
03
04 def houghLines(src, rho, theta, thresh):                        # 허프 변환 함수
05     acc_mat = accumulate(src, rho, theta)                      # 직선 누적 행렬 계산
06     acc_dst = masking(acc_mat, 7, 3, thresh)                   # 마스크 처리 - 7행, 3열
07     lines = select_lines(acc_dst, rho, theta, thresh)          # 임계 직선 선택
08     return lines
09
10 def draw_houghLines(src, lines, nline):                        # 검출 직선 그리기 함수
11     dst = cv2.cvtColor(src, cv2.COLOR_GRAY2BGR)               # 컬러 영상 변환
12     min_length = min(len(lines), nline)
13
14     for i in range(min_length):
15         rho, radian = lines[i, 0, 0:2]                         # 수직 거리, 각도 - 3차원 행렬임
16         a, b = math.cos(radian), math.sin(radian)
17         pt= (a * rho, b * rho)                                  # 검출 직선상의 한 좌표
18         delta= (-1000 * b, 1000 * a)                           # 직선상의 이동 위치
19         pt1 = np.add(pt, delta).astype('int')
20         pt2 = np.subtract(pt, delta).astype('int')
21         cv2.line(dst, tuple(pt1), tuple(pt2), (0, 255, 0), 2, cv2.LINE_AA)
22
23     return dst
24
```

허프변환
전체 과정 수행 함수

직선위 2개 좌표 계산

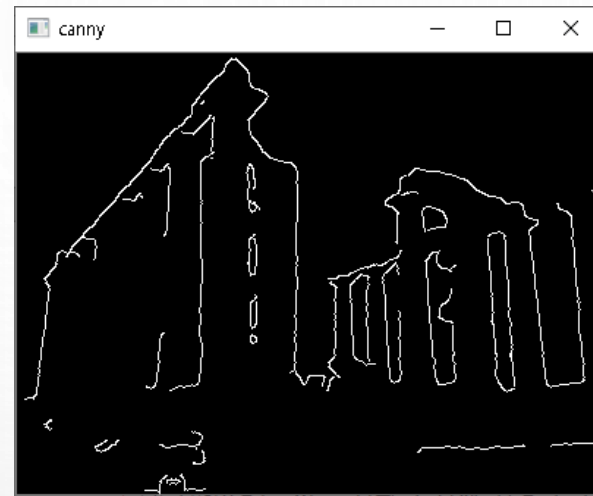
10.1.7 최종 완성 프로그램

❖ 영상에 검출 직선 그리기 함수

```
25 image = cv2.imread("images/hough.jpg", cv2.IMREAD_GRAYSCALE)
26 if image is None: raise Exception("영상파일 읽기 에러")
27 blur = cv2.GaussianBlur(image, (5, 5), 2, 2)           # 가우시안 블러링
28 canny = cv2.Canny(blur, 100, 200, 5)                  # 캐니 에지 추출
29
30 rho, theta = 1, np.pi / 180                          # 수직거리 간격, 각도 간격
31 lines1 = houghLines(canny, rho, theta, 80)             # 저자 구현 함수
32 lines2 = cv2.HoughLines(canny, rho, theta, 80)         # OpenCV 함수
33 dst1 = draw_houghLines(canny, lines1, 7)               # 직선 그리기
34 dst2 = draw_houghLines(canny, lines2, 7)
35
36 cv2.imshow("image", image)
37 cv2.imshow("canny", canny);
38 cv2.imshow("detected lines", dst1)
39 cv2.imshow("detected lines_OpenCV", dst2)
40 cv2.waitKey(0)
```


10.1.7 최종 완성 프로그램

◆ 실행결과



■ 허프 변환에 의한 선분 검출

```
cv2.HoughLines(image, rho, theta, threshold, lines=None, srn=None, stn=None,  
               min_theta=None, max_theta=None) -> lines
```

- image: 입력 에지 영상
- rho: 축적 배열에서 rho 값의 간격. (e.g.) 1.0 → 1픽셀 간격.
- theta: 축적 배열에서 theta 값의 간격. (e.g.) $\text{np.pi} / 180 \rightarrow 1^\circ$ 간격.
- threshold: 축적 배열에서 직선으로 판단할 임계값
- lines: 직선 파라미터(rho, theta) 정보를 담고 있는 `numpy.ndarray`.
`shape=(N, 1, 2)`. `dtype=numpy.float32`.
- srn, stn: 멀티 스케일 허프 변환에서 rho 해상도, theta 해상도를 나누는 값.
기본값은 0이고, 이 경우 일반 허프 변환 수행.
- min_theta, max_theta: 검출할 선분의 최소, 최대 theta 값

■ 확률적 허프 변환에 의한 선분 검출

```
cv2.HoughLinesP(image, rho, theta, threshold, lines=None,  
                 minLineLength=None, maxLineGap=None) -> lines
```

- image: 입력 에지 영상
- rho: 축적 배열에서 rho 값의 간격. (e.g.) 1.0 → 1픽셀 간격.
- theta: 축적 배열에서 theta 값의 간격. (e.g.) $\text{np.pi} / 180 \rightarrow 1^\circ$ 간격.
- threshold: 축적 배열에서 직선으로 판단할 임계값
- lines: 선분의 시작과 끝 좌표(x1, y1, x2, y2) 정보를 담고 있는 `numpy.ndarray`.
shape=(N, 1, 4). dtype=`numpy.int32`.
- minLineLength: 검출할 선분의 최소 길이
- maxLineGap: 직선으로 간주할 최대 에지 점 간격

Hough_lines.py

■ 확률적 허프 변환 직선 검출 예제

```
src = cv2.imread('building.jpg', cv2.IMREAD_GRAYSCALE)

edges = cv2.Canny(src, 50, 150)

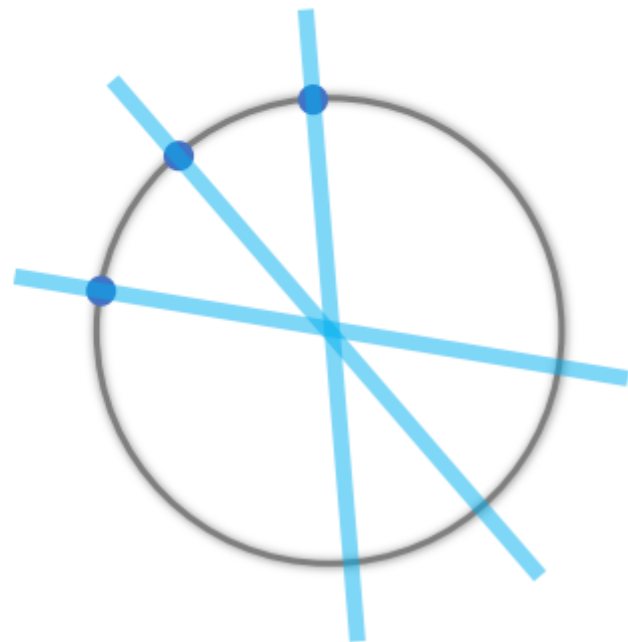
lines = cv2.HoughLinesP(edges, 1.0, np.pi / 180., 160,
                        minLineLength=50, maxLineGap=5)

dst = cv2.cvtColor(edges, cv2.COLOR_GRAY2BGR)

if lines is not None:
    for i in range(lines.shape[0]):
        pt1 = (lines[i][0][0], lines[i][0][1]) # 시작점 좌표
        pt2 = (lines[i][0][2], lines[i][0][3]) # 끝점 좌표
        cv2.line(dst, pt1, pt2, (0, 0, 255), 2, cv2.LINE_AA)
```

원 검출 가능?

- 허프 변환을 응용하여 원을 검출할 수 있음
 - 원의 방정식: $(x - a)^2 + (y - b)^2 = c^2 \rightarrow$ 3차원 축적 평면?
- 속도 향상을 위해 Hough gradient method 사용
 - 입력 영상과 동일한 2차원 평면 공간에서 축적 영상을 생성
 - 에지 픽셀에서 그래디언트 계산
 - 에지 방향에 따라 직선을 그리면서 값을 누적
 - 원의 중심을 먼저 찾고, 적절한 반지름을 검출
 - 단점
 - 여러 개의 동심원을 검출 못함
 $\rightarrow$ 가장 작은 원 하나만 검출됨



■ 허프 변환 원 검출 함수

```
cv2.HoughCircles(image, method, dp, minDist, circles=None, param1=None,  
                 param2=None, minRadius=None, maxRadius=None) -> circles
```

- image: 입력 영상. (에지 영상이 아닌 일반 영상)
- method: OpenCV 4.2 이하에서는 `cv2.HOUGH_GRADIENT`만 지정 가능
- dp: 입력 영상과 축적 배열의 크기 비율. 1이면 동일 크기.
2이면 축적 배열의 가로, 세로 크기가 입력 영상의 반.
- minDist: 검출된 원 중심점들의 최소 거리
- circles: (cx, cy, r) 정보를 담은 `numpy.ndarray`. shape=(1, N, 3), dtype=np.float32.
- param1: Canny 에지 검출기의 높은 임계값
- param2: 축적 배열에서 원 검출을 위한 임계값
- minRadius, maxRadius: 검출할 원의 최소, 최대 반지름

Hough_circles.py

■ 허프 변환 원 검출 예제

```
src = cv2.imread('dial.jpg')
gray = cv2.cvtColor(src, cv2.COLOR_BGR2GRAY)
blr = cv2.GaussianBlur(gray, (0, 0), 0.5) ← cv2.Hough_GRADIENT 방법 사용 시
                                             블러링 권장

def on_trackbar(pos):
    rmin = cv2.getTrackbarPos('minRadius', 'img')
    rmax = cv2.getTrackbarPos('maxRadius', 'img')
    th = cv2.getTrackbarPos('threshold', 'img')

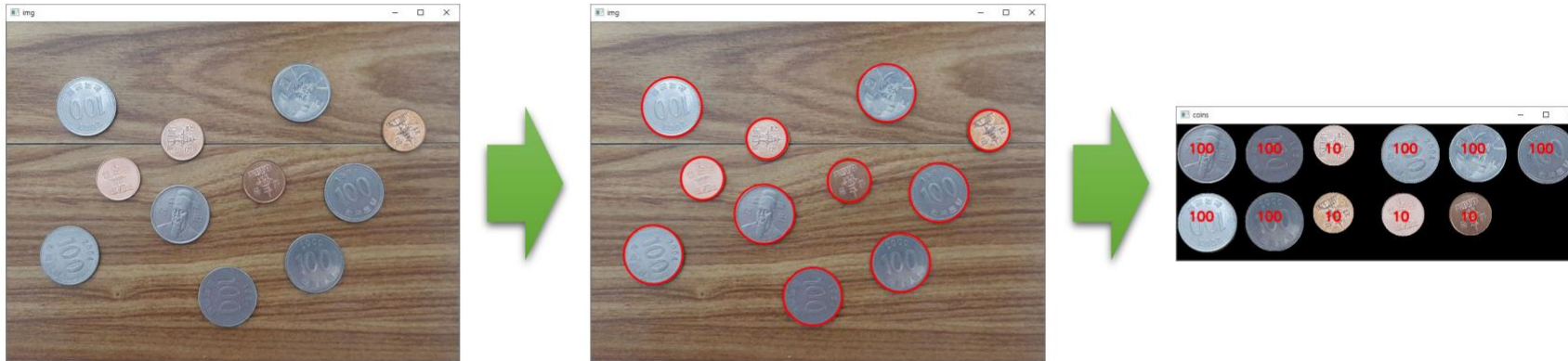
    circles = cv2.HoughCircles(blr, cv2.HOUGH_GRADIENT, 1, 50,
                               param1=120, param2=th, minRadius=rmin, maxRadius=rmax)

    dst = src.copy()
    if circles is not None:
        for i in range(circles.shape[1]):
            cx, cy, radius = np.uint16(circles[0][i])
            cv2.circle(dst, (cx, cy), radius, (0, 0, 255), 2, cv2.LINE_AA)
```

심화 예제(동전 세기)

■ 동전 카운터

- 영상의 동전을 검출하여 금액이 얼마인지를 자동으로 계산하는 프로그램
- 편의상 동전은 100원짜리와 10원짜리만 있다고 가정



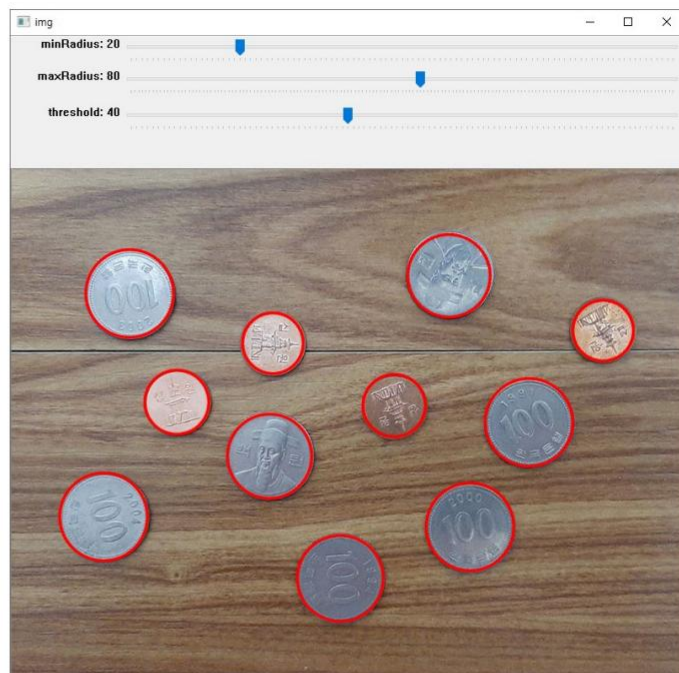
■ 구현할 기능

- 동전 검출하기 → 허프 원 검출
- 동전 구분하기 → 색상 정보 이용

심화 예제(동전 세기)

■ 동전 검출하기

- 동그란 객체는 동전만 있다고 가정
→ cv2.HoughCircles() 함수 사용
- 영상 크기: 800x600 (px)
- 동전 크기
 - 100원: 약 100x100 (px)
 - 10원: 약 80x80 (px)



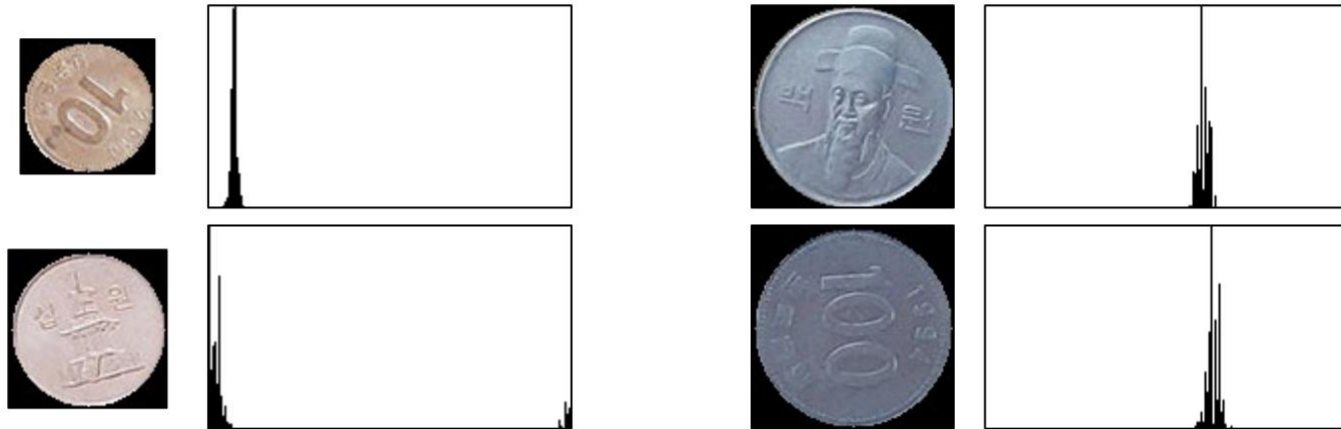
```
src = cv2.imread('coins2.jpg')
gray = cv2.cvtColor(src, cv2.COLOR_BGR2GRAY)
blr = cv2.GaussianBlur(gray, (0, 0), 1)

circles = cv2.HoughCircles(blr, cv2.HOUGH_GRADIENT, 1, 50,
                           param1=150, param2=40, minRadius=20, maxRadius=80)
```

심화 예제(동전 세기)

■ 동전 구분하기

- 동전 영역 부분 영상 추출 → HSV 색 공간으로 변환
- 동전 영역에 대해서만 Hue 색 성분 분포 분석



- 동전 영역 픽셀에 대해 Hue 값을 +40만큼 시프트하고, Hue 평균을 분석
 - Hue 평균이 90보다 작으면 10원
 - Hue 평균이 90보다 크면 100원

심화 예제(동전 세기)

■ 동전 구분하기

